

Java Developer (Fresher) Technical Interview Handbook

PART 5 of 5 | Spring Boot, REST APIs, Hibernate/JPA & Maven

- Part 1 - Core Java Fundamentals & OOP (delivered)
- Part 2 - Collections, Exception Handling & Multithreading (delivered)
- Part 3 - JDBC & SQL / MySQL (delivered)
- Part 4 - HTML, CSS, JavaScript basics & Git (delivered)
- Part 5 - Spring Boot, REST APIs, Hibernate/JPA & Maven (this document - final part)

Note: the JD lists these as "preferred/optional" skills, but interviewers often use them to see how far beyond the minimum requirements a candidate has explored - since you already have hands-on project experience with these, this part goes a little deeper than a bare-minimum fresher answer would.

Section A: Spring Framework & Spring Boot

Q1. What is the Spring Framework, and what problem does it solve?

Spring is a comprehensive framework for building Java applications, built around the core idea of Inversion of Control (IoC) and Dependency Injection (DI). Instead of my classes creating and managing their own dependencies directly (tightly coupling them together), Spring's IoC Container creates and manages all the objects (called 'beans') and automatically supplies each class with whatever dependencies it needs.

This solves the problem of tight coupling and makes applications far easier to test (I can substitute a mock dependency in tests), maintain, and extend, since classes don't need to know HOW their dependencies are created - only that they'll be provided.

Q2. What is Dependency Injection (DI) and Inversion of Control (IoC)? How are they related?

Inversion of Control is a general design PRINCIPLE - instead of a class controlling the creation and lifecycle of the objects it depends on, that control is handed over ('inverted') to an external container/framework.

Dependency Injection is the specific TECHNIQUE Spring uses to implement IoC - the required dependencies are 'injected' into a class from outside, rather than the class instantiating them itself with 'new'. Spring supports constructor injection (passing dependencies through the constructor - generally the recommended approach, since it makes dependencies explicit and allows fields to be made final), setter injection (through setter methods), and field injection (directly on the field using @Autowired, which is convenient but generally discouraged for production code since it makes testing and immutability harder).

```
@Service
public class JobService {
    private final JobRepository jobRepository; // dependency

    @Autowired // constructor injection - recommended approach
    public JobService(JobRepository jobRepository) {
        this.jobRepository = jobRepository;
    }
}
```

Q3. What is the difference between Spring and Spring Boot?

Spring is the underlying framework - powerful, but it traditionally required a lot of manual, verbose XML or Java-based configuration to set up things like the web server, database connections, and bean definitions before you could even start writing business logic.

Spring Boot is built ON TOP OF Spring, and its whole purpose is to make Spring applications much faster and easier to set up, through 'auto-configuration' (it intelligently configures beans based on what's on the classpath), 'starter' dependencies (pre-bundled sets of commonly-used dependencies, like spring-boot-starter-web), and an embedded server (like Tomcat) bundled directly into the application, so there's no need to separately deploy a WAR file to an external server - I can just run the application directly as an executable JAR.

Q4. What is the purpose of the @SpringBootApplication annotation?

`@SpringBootApplication` is a convenience annotation placed on the main class that combines three separate annotations into one: `@Configuration` (marks this class as a source of bean definitions), `@EnableAutoConfiguration` (tells Spring Boot to automatically configure beans based on the dependencies present on the classpath - for example, if `spring-boot-starter-web` is present, it auto-configures an embedded Tomcat server and Spring MVC), and `@ComponentScan` (tells Spring to scan the current package and its sub-packages for other Spring-managed components like `@Service`, `@Repository`, and `@Controller` classes, so they get automatically registered as beans).

```
@SpringBootApplication
public class JobPortalApplication {
    public static void main(String[] args) {
        SpringApplication.run(JobPortalApplication.class, args);
    }
}
```

Q5. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

All four are 'stereotype' annotations that mark a class as a Spring-managed bean, to be automatically detected by component scanning and registered in the IoC container. Functionally, they all do the same underlying thing, but they exist as separate annotations to clearly communicate INTENT and layer, and some add extra specialized behavior.

`@Component` is the generic, base-level annotation for any Spring-managed bean that doesn't fit a more specific category.

`@Service` marks a class in the service/business-logic layer - it's semantically meant to hold business logic, though it behaves the same as `@Component` under the hood.

`@Repository` marks a class in the data-access (DAO) layer. It has one important extra feature: Spring automatically translates database-specific exceptions thrown by that class into Spring's own consistent `DataAccessException` hierarchy, which makes exception handling more uniform regardless of which database technology is used underneath.

`@Controller` (and its specialization `@RestController`) marks a class in the web/presentation layer, responsible for handling incoming HTTP requests.

Q6. What is the difference between `@Controller` and `@RestController`?

`@Controller` is the traditional Spring MVC annotation for a class that handles web requests and typically returns the NAME of a view (like a Thymeleaf or JSP template) to be rendered as an HTML page.

`@RestController` is a convenience annotation that combines `@Controller` with `@ResponseBody`.

`@ResponseBody` tells Spring that the return value of each method should be written DIRECTLY into the HTTP response body (typically automatically converted to JSON), rather than being interpreted as a view name. This is why `@RestController` is the standard choice for building REST APIs that return data (like JSON) instead of rendering full HTML pages.

```
@RestController
@RequestMapping("/api/jobs")
public class JobController {

    @GetMapping("/{id}")
    public Job getJob(@PathVariable Long id) {
        return jobService.findById(id); // returned directly as JSON
    }
}
```

```
}  
}
```

Q7. What is application.properties (or application.yml) used for in Spring Boot?

It's the central configuration file where I set values that control the application's behavior WITHOUT changing any Java code - things like the server port, database connection URL/username/password, logging levels, and various framework-specific settings (like Hibernate's ddl-auto behavior).

application.yml serves the exact same purpose but uses YAML's indentation-based format instead of flat key=value pairs, which can be more readable for deeply nested configuration - either format works, and it's mostly a matter of preference which one a project uses.

```
# application.properties  
server.port=8081  
spring.datasource.url=jdbc:mysql://localhost:3306/jobportal  
spring.datasource.username=root  
spring.datasource.password=secret  
spring.jpa.hibernate.ddl-auto=update
```

Q8. What are Spring Boot 'starters'? Give an example.

Starters are pre-configured dependency bundles that group together all the libraries commonly needed for a particular kind of functionality, so I don't have to manually figure out and add each individual dependency and worry about version compatibility between them.

For example, adding spring-boot-starter-web to my pom.xml pulls in everything needed to build a web application/REST API - Spring MVC, an embedded Tomcat server, and Jackson for JSON conversion - all in one line. Similarly, spring-boot-starter-data-jpa bundles Hibernate and Spring Data JPA together for database access.

Section B: REST APIs

Q9. What is REST, and what are its key principles?

REST (Representational State Transfer) is an architectural style for designing web services/APIs, built around the idea of treating everything as a 'resource' (like a job, a user, or an application) identified by a URL, and using standard HTTP methods to perform operations on those resources.

The key principles are: statelessness - each request from the client must contain all the information needed to process it; the server doesn't store any client session state between requests. Client-server separation - the frontend and backend can evolve independently as long as the API contract stays consistent. Uniform interface - resources are accessed and manipulated using a consistent, standard set of HTTP methods and status codes. Resource-based URLs - endpoints represent nouns/resources (like /jobs or /jobs/5), not actions (avoid something like /getJob).

Q10. What are the common HTTP methods used in REST APIs, and what does each one mean?

GET retrieves a resource (or a list of resources) - it should be safe and not change any data. POST creates a new resource. PUT updates/replaces an existing resource ENTIRELY (you send the full updated object). PATCH partially updates a resource (you send only the fields that changed). DELETE removes a resource.

GET, PUT, and DELETE are also expected to be 'idempotent' - calling them multiple times with the same input produces the same end result/state as calling them once (deleting the same resource twice still just results in it being gone). POST is generally NOT idempotent, since calling it twice typically creates two separate resources.

GET	/api/jobs	-> list all jobs
GET	/api/jobs/5	-> get job with id 5
POST	/api/jobs	-> create a new job
PUT	/api/jobs/5	-> replace job 5 entirely
DELETE	/api/jobs/5	-> delete job 5

Q11. What are some common HTTP status codes, and what do they mean?

Status codes are grouped by their first digit. 2xx means success: 200 OK (general success), 201 Created (a new resource was successfully created, typically returned after a POST), 204 No Content (success, but there's nothing to return in the body, common after a DELETE).

3xx means redirection, less commonly discussed at fresher level. 4xx means the CLIENT made an error: 400 Bad Request (malformed request/invalid input), 401 Unauthorized (the client isn't authenticated - no valid credentials provided), 403 Forbidden (the client IS authenticated, but doesn't have permission for this action), 404 Not Found (the requested resource doesn't exist), 409 Conflict (the request conflicts with the resource's current state, like trying to create a duplicate).

5xx means the SERVER made an error: 500 Internal Server Error is the general catch-all for an unexpected failure on the server side.

Interview tip: Interviewers commonly ask you to distinguish 401 vs 403 specifically - 401 means 'I don't know who you are' (missing/invalid login), 403 means 'I know who you are, but you're not allowed to do this'.

Q12. What is the difference between @PathVariable and @RequestParam?

Both are used to extract values from an incoming HTTP request in a Spring controller method, but from different parts of the URL.

`@PathVariable` extracts a value that's embedded directly as PART of the URL path itself, typically used to identify a specific resource, like the id in `/api/jobs/{id}`.

`@RequestParam` extracts a value from the QUERY STRING (the part after a '?' in the URL, like `?status=active&page=2`), typically used for optional filters, search terms, pagination, or sorting options.

```
// GET /api/jobs/5?status=active
@GetMapping("/jobs/{id}")
public Job getJob(
    @PathVariable Long id,           // 5
    @RequestParam String status) {   // "active"
    ...
}
```

Q13. What is the purpose of `@RequestBody` and `@ResponseBody`?

`@RequestBody` is placed on a controller method's parameter to tell Spring to take the incoming HTTP request's body (typically JSON) and automatically convert/deserialize it into a Java object, using a library like Jackson under the hood.

`@ResponseBody` is placed on a method (or implied automatically by `@RestController`) to tell Spring to take the RETURN value of that method and serialize it directly into the HTTP response body as JSON, instead of treating the returned value as the name of a view to render.

```
@PostMapping("/jobs")
public Job createJob(@RequestBody Job newJob) {
    // newJob is automatically built from the incoming JSON request body
    return jobService.save(newJob); // automatically converted back to JSON in the
    response
}
```

Q14. How would you design a simple REST API for managing job postings? (system-design style question)

I'd design the endpoints around the 'jobs' resource, using nouns and standard HTTP methods rather than verbs in the URL: GET `/api/jobs` to list all jobs (with optional query parameters for filtering/pagination), GET `/api/jobs/{id}` to get one specific job, POST `/api/jobs` to create a new job posting, PUT `/api/jobs/{id}` to update an existing one, and DELETE `/api/jobs/{id}` to remove it.

For related sub-resources, like applications submitted to a specific job, I'd nest the URL, like GET `/api/jobs/{id}/applications`. I'd also make sure to return appropriate status codes for each case (201 on successful creation, 404 if a job id doesn't exist, 400 for invalid input), and use DTOs (Data Transfer Objects) rather than exposing my JPA entity classes directly, to control exactly what data the API exposes and avoid issues like lazy-loading exceptions during JSON serialization.

Interview tip: If you've actually built something like this in your job portal project, mention it concretely here - real project experience is a strong differentiator even for a fresher-level interview.

Section C: Hibernate & JPA

Q15. What is ORM, and what problem does Hibernate solve?

ORM (Object-Relational Mapping) is a technique that maps Java objects/classes directly to database tables, and object fields to table columns, so I can work with plain Java objects in my code instead of manually writing SQL and dealing with ResultSets everywhere.

Hibernate is a popular ORM framework/implementation for Java. It solves the problem of writing large amounts of repetitive, boilerplate JDBC code (connections, PreparedStatements, manually mapping ResultSet columns to object fields) - instead, Hibernate handles generating the underlying SQL, managing the object-to-table mapping, and tracking changes to objects automatically, letting me interact with the database mostly through simple object method calls like `session.save(employee)`.

Q16. What is the difference between JPA and Hibernate?

JPA (Java Persistence API) is a SPECIFICATION - a set of interfaces and rules (defined by Java/Jakarta EE) that describe HOW object-relational mapping should work in Java, but it doesn't provide an actual implementation itself.

Hibernate is an actual IMPLEMENTATION of the JPA specification (it's also the most widely used one), along with some additional Hibernate-specific features beyond what plain JPA defines. Other JPA implementations exist too, like EclipseLink.

In practice, most Spring Boot projects code against the JPA interfaces (like `EntityManager`, `JpaRepository`) rather than Hibernate-specific classes directly, which means the underlying ORM provider could theoretically be swapped out without changing much application code - though in reality, Hibernate is by far the default and most common choice.

Q17. What is an Entity in JPA/Hibernate? Explain @Entity, @Id, and @GeneratedValue.

An Entity is a plain Java class that represents (is mapped to) a table in the database - each instance of the class corresponds to one row in that table, and each field typically corresponds to a column.

`@Entity` marks a class as a JPA entity, telling Hibernate to manage it and map it to a database table (by default, the table name matches the class name, though this can be customized with `@Table`).

`@Id` marks the field that serves as the entity's primary key.

`@GeneratedValue` specifies HOW the primary key's value should be automatically generated - common strategies include `IDENTITY` (relies on the database's own auto-increment column), `SEQUENCE` (uses a database sequence object, common in databases like PostgreSQL/Oracle), and `AUTO` (lets Hibernate pick an appropriate strategy based on the underlying database).

```
@Entity
@Table(name = "jobs")
public class Job {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String company;
}
```

Q18. What is the difference between JpaRepository, CrudRepository, and PagingAndSortingRepository in Spring Data JPA?

These form a hierarchy of interfaces, each extending the previous one and adding more functionality, and in practice I don't need to write an implementation for any of them - Spring Data JPA generates the implementation automatically at runtime just from the interface definition.

CrudRepository provides the basic CRUD operations - save(), findById(), findAll(), deleteById(), and so on.

PagingAndSortingRepository extends CrudRepository and adds support for pagination and sorting results.

JpaRepository extends PagingAndSortingRepository and adds a few more JPA-specific conveniences, like flush() and batch-delete operations. In most Spring Boot projects, JpaRepository is simply used by default since it includes everything the other two offer.

```
public interface JobRepository extends JpaRepository<Job, Long> {  
    // custom query method - Spring generates the implementation from the method name  
    List<Job> findByCompany(String company);  
}
```

Q19. What is the difference between lazy loading and eager loading in Hibernate?

These control WHEN related/associated data is fetched from the database, relevant whenever an entity has a relationship to another entity (like @OneToMany or @ManyToOne).

Eager loading (FetchType.EAGER) fetches the related data IMMEDIATELY, at the same time as the main entity - for example, loading a Job and its list of Applications together in one go (often via a JOIN).

Lazy loading (FetchType.LAZY) delays fetching the related data until it's actually ACCESSED in code for the first time - the main entity loads quickly on its own, and Hibernate only queries for the related data if/when that association is actually used.

Lazy loading is generally preferred for performance, especially for collections, since eagerly loading large or unnecessary related data on every single query wastes resources. However, lazy loading introduces a well-known pitfall: if you try to access a lazily-loaded association AFTER the database session/transaction has already closed, it throws a LazyInitializationException, since Hibernate can no longer go back to the database to fetch it.

Interview tip: Mentioning LazyInitializationException by name and explaining WHY it happens (session already closed) shows real hands-on debugging experience, not just textbook knowledge.

Q20. What are the different types of entity relationships/mappings in JPA?

@OneToOne maps a relationship where one entity is associated with exactly one instance of another - like a User having exactly one Profile.

@OneToMany / @ManyToOne together map a relationship where one entity can be associated with MANY instances of another - like one Job having many Applications (Job side is @OneToMany, Application side is @ManyToOne, since many applications belong to one job).

@ManyToMany maps a relationship where many instances of one entity relate to many instances of another - like Students and Courses, where a student can enroll in many courses and a course can have many students. This requires an underlying join table in the database to represent the association.

```

@Entity
public class Job {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "job")
    private List<Application> applications;
}

@Entity
public class Application {
    @Id @GeneratedValue
    private Long id;

    @ManyToOne
    @JoinColumn(name = "job_id")
    private Job job;
}

```

Q21. What is the N+1 select problem in Hibernate, and how can it be avoided?

The N+1 problem happens when I fetch a list of N parent entities with a single query, but then, for each one, Hibernate lazily fires off a SEPARATE additional query to fetch its related child data - resulting in 1 (for the parent list) + N (one per parent) total queries instead of just one efficient query, which can badly hurt performance as the dataset grows.

For example, fetching 100 jobs and then accessing each job's list of applications individually in a loop would trigger 1 query for the jobs, plus 100 more queries, one for each job's applications.

It can be avoided using JOIN FETCH in a JPQL query, which tells Hibernate to fetch the parent and its related data together in a SINGLE query using a SQL join, instead of triggering separate lazy-loaded queries later.

```

// Causes N+1: applications fetched lazily, one query per job
@Query("SELECT j FROM Job j")
List<Job> findAllJobs();

// Fixed: fetches jobs and their applications together in one query
@Query("SELECT j FROM Job j JOIN FETCH j.applications")
List<Job> findAllJobsWithApplications();

```

Section D: Maven

Q22. What is Maven, and what problem does it solve?

Maven is a build automation and project/dependency management tool for Java projects. It solves two main problems: dependency management - instead of manually downloading .jar files for every library my project needs and figuring out compatible versions myself, I just declare what I need in a configuration file, and Maven automatically downloads it (and any of ITS dependencies too) from a central repository. And standardized build process - Maven defines a consistent way to compile, test, package, and deploy a project, so any Maven project follows a predictable structure and set of commands, making it much easier to build on different machines or CI/CD pipelines.

Q23. What is pom.xml, and what are its key sections?

pom.xml (Project Object Model) is Maven's central configuration file, located at the root of the project. It defines everything Maven needs to know about the project.

Key sections include: groupId, artifactId, and version, which together uniquely identify the project (and are used when this project is itself published as a dependency for others); dependencies, listing all the external libraries the project needs, each with its own groupId/artifactId/version; properties, for reusable values like the Java version being targeted; build/plugins, for configuring the build process itself, like the Spring Boot Maven plugin that packages the application into an executable JAR; and parent, commonly used in Spring Boot projects to inherit shared, sensible default configuration from spring-boot-starter-parent.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Q24. What are the main phases of the Maven build lifecycle?

Maven's default lifecycle consists of a sequence of phases, and running any given phase automatically runs all the phases before it too. The main ones, in order, are: validate (checks the project structure/pom.xml are correct), compile (compiles the source code), test (runs unit tests using a testing framework like JUnit), package (bundles the compiled code into a distributable format, like a JAR or WAR), verify (runs checks on the results of integration tests, if any), install (installs the packaged artifact into my local Maven repository, so other local projects can use it as a dependency), and deploy (copies the final package to a remote repository for sharing with other developers or environments).

```
mvn clean install
# clean -> deletes previous build output (target/ folder)
# install -> runs validate through install phases in order
```

Q25. What is the difference between Maven and Gradle?

Both are build automation and dependency management tools for Java (and other JVM languages), but they differ mainly in configuration style and performance.

Maven uses an XML-based configuration file (`pom.xml`), following a fixed, convention-based lifecycle. It's very widely adopted, especially in enterprise Java projects, and has a huge ecosystem.

Gradle uses a more concise, code-based configuration (Groovy or Kotlin DSL, `build.gradle`), which is generally considered more flexible and readable than XML, and it's known for better build performance through features like incremental builds and build caching. Gradle is the default and required build tool for Android app development.

For a fresher role, it's usually enough to know Maven's basics well and be aware that Gradle exists as a commonly used alternative - most job-portal-style enterprise Spring Boot projects (like the one you're likely to be asked about) still commonly use Maven.